

Software Architecture
(T630019402)

Lecture #5, March 10, 2026

Jukka Ruohonen

Case #4.A Presentations

- ▶ Investigate the *syzkaller/syzbot* (as also done by Ruohonen & Rindell 2019) used to continuously fuzz the Linux kernel:
<https://syzkaller.appspot.com>
- ▶ Can you say anything about the **invalid cases**?
- ▶ What can you say by investigating the **graphing functionality** provided on the dashboard, including with respect to fuzzing, open bugs, and coverage?
 - Presenting graphs would be nice but not mandatory
- ▶ Are all **subsystems** tested equally well?
- ▶ Has the kernel's "**health**" been stable in overall?
- ▶ Any other insights you might want to share?

Case #4.B Presentations

- ▶ **CrowdStrike** had a major incident in July 2024 and then **Cloudflare** in December 2025
- ▶ Read Internet sources about these incidents with respect to:
 - What were the **causes**?
 - What were the **effects**?
 - What do the incidents say about **testability**?
 - How about **deployability**?
 - What can be **learned** from the incidents?

Today's Lecture

- ▶ Today's lecture is about **availability** as a distinct QA
- ▶ As already the case #4.B indicates, failing to ensure availability may have severe consequences, but there are many distinct ways to frame availability and its violations
 - Therefore, some of the topics will be revisited in later lectures addressing performance and **scalability** in particular

Question

How would you characterize availability?

Framing Availability (1/6)

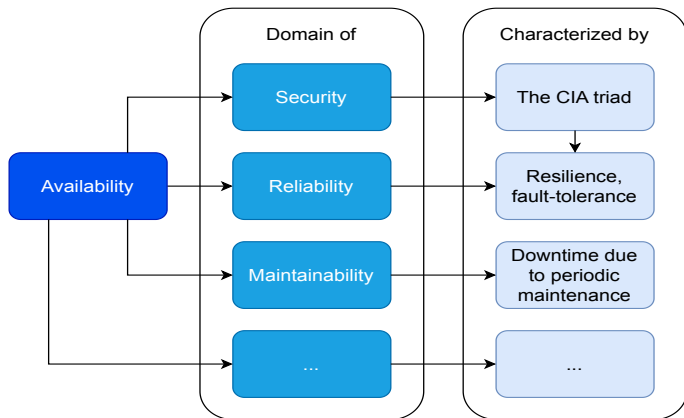


Figure 1: What Other Domains Might Characterize Availability?

Framing Availability (2/6)

- ▶ Availability generally means that a system is available to serve, use, deliver, or otherwise function whenever needed
 - A **security** perspective to availability adds that the system should be available for **authorized** parties only; in addition, there are again **trade-offs** between security and reliability
- ▶ From a **reliability** perspective, availability is related to the concept of a system **failure**, which is *a deviation of the system from its specification* (Bass et al. 2021)
 - Another closely related reliability concept is **fault**, which can be defined as the *internal or external cause of a given failure*

Framing Availability (3/6)

- ▶ With the previous reliability concepts, availability can be seen as the ability of a system to **prevent**, **mask**, and **repair** faults, such that they do not become actual failures
- ▶ However, given the previous definition of a failure as a deviation from expected behavior, preventing, masking, and repairing faults require understanding about **outage periods that exceed a required availability guarantees**
 - Therefore, again, recall the measurability of QAs

Framing Availability (4/6)

Table 1: Example Requirements for Availability (Bass et al. 2021)

Availability	Downtime / 90 days	Downtime / year
90.0%	21 hours, 36 minutes	3 days, 15.6 hours
90.9%	2 hours, 10 minutes	8 hours, 46 seconds
99.99%	12 minutes, 58 seconds	52 minutes, 34 seconds
99.999%	1 minute, 18 seconds	5 minutes, 15 seconds
99.9999%	8 seconds	32 seconds

- ▶ Which would you consider as a sensible requirement for a general-purpose high-availability system?

Framing Availability (5/6)

- ▶ More specifically, faults can be **prevented** (cf. good development practices and tools), **removed** (cf. fixing bugs), **tolerated** (cf. accept these as residual risks), and **forecast**
 - With these broad techniques, a system can achieve **fault-tolerance**, which is broadly what **resilience** is about
- ▶ Relevant questions include (but are not limited to): *how to detect* faults and *how to prevent these*; *how frequent* faults may be; *what happens* in case a fault triggers; *how long* can a system be unavailable; *how to notify* both faults and failures; and *how to recover* from these?

Framing Availability (6/6)

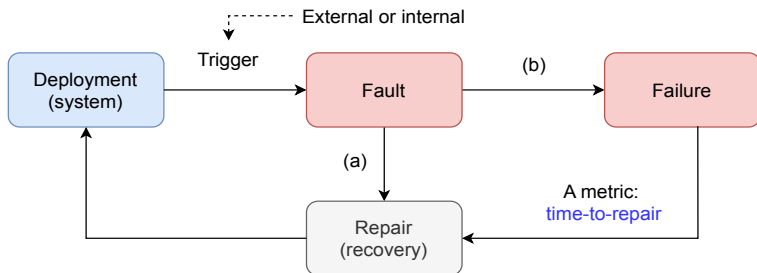


Figure 2: When Building Fault-Tolerant Systems, We Should Seek to Maximize the Probability for the Route (a), But If We End Up With the Route (b), We Should Seek to Minimize the Time-to-Repair Metric

Faults, So Many Faults

- ▶ **Hardware:** *disks, memory errors, CPU failures, power cuts, network equipment malfunction, natural disasters, and so on*
- ▶ **Software:** *bugs, memory leaks, deadlocks, infinite loops, ...*
- ▶ **Connectors:** *extensive latency, dropped packets, corrupted packets, fragmented packets, unreachable paths, etc.*
- ▶ **Operational failures:** *configuration errors, risk analysis failures, planning failures, and other human errors, etc.*
- ▶ **Systemic failures:** *escalations and cascades; even full-blown crises rather than mere failures*

Availability Scenarios

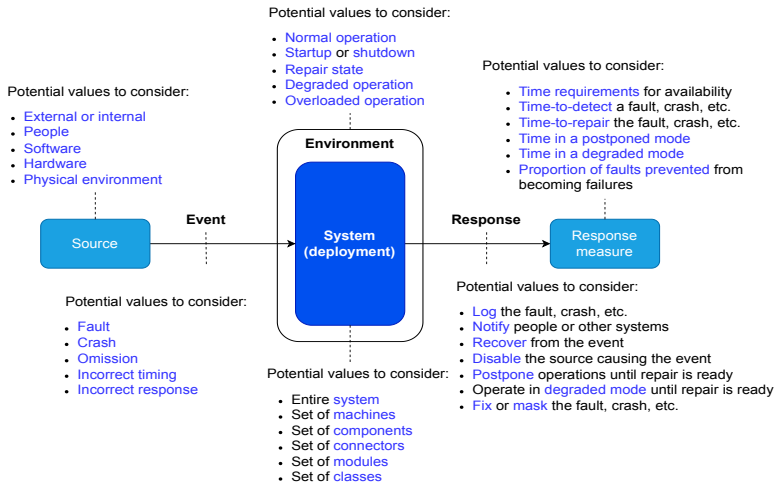


Figure 3: Adopted and modified from Bass et al. (2021)

Metrics (1/2)

- ▶ Although the time-to-repair was noted earlier, numerous other metrics are also possible, as seen from the previous figure
- ▶ Note that **time-to-detect** may be short in the availability context because the events have almost always visible consequences for users and others
 - However, when moving beyond availability alone, time-to-detect may be long; for instance, surveys indicate that even circa 100 days is required on average for organizations to detect data breaches (Ruohonen et al. 2024)
- ▶ From a fault-tolerance perspective, the perhaps most important metric is related to the **proportion of faults** detected but prevented from becoming actual failures

Metrics (2/2)

- ▶ Although their applicability to software remains debated, classical **reliability** (forecasting) **models** assume that the growth rates of faults discovered decrease over time

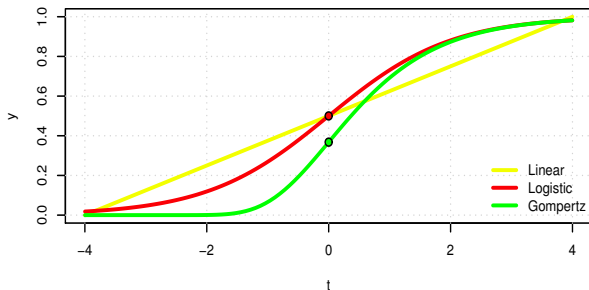


Figure 4: Adopted and modified from Ruohonen et al. (2025, Fig. 3)

Availability Tactics

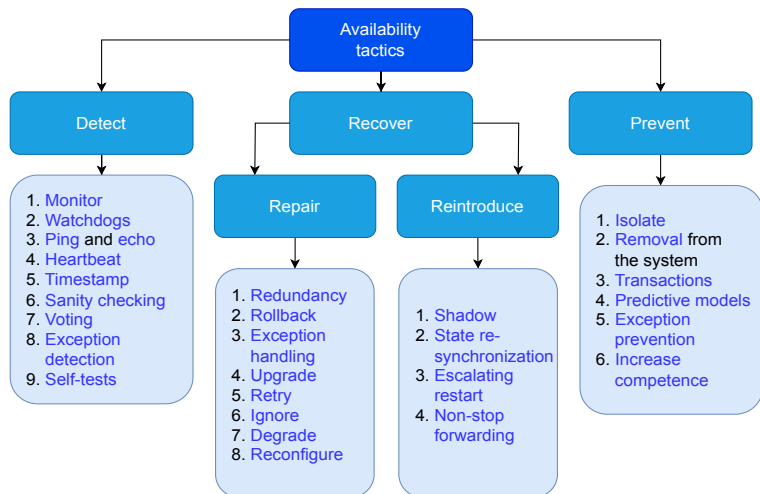


Figure 5: Tactics for Availability (adopted from Bass et al. 2021)

Availability Tactics: Detect $\mapsto$ Monitor

- ▶ **Monitoring** is the overall tactic for detecting availability transgressions; a system's “health” is continuously monitored, whether in terms of input/output operations, memory consumption, CPU capacity used, or network load
- ▶ If a fault or other malfunction is detected, a monitoring component can initiate different repair functions
- ▶ There is a **trade-off** with performance

Availability Tactics: Detect $\mapsto$ Watchdogs and Heartbeats

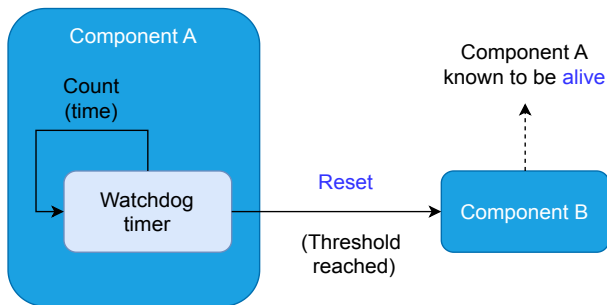


Figure 6: Watchdog Timers in a Nutshell (and, by extension, heartbeats)

Availability Tactics: Detect $\mapsto$ Ping and Echo (1/2)

```
adorno: ~/.../software_architecture/doc x adorno: ~
jruoho@adorno~$ ping sdu.dk
PING sdu.dk (52.233.201.66) 56(84) bytes of data.
64 bytes from 52.233.201.66: icmp_seq=1 ttl=112 time=17.4 ms
64 bytes from 52.233.201.66: icmp_seq=2 ttl=112 time=19.6 ms
64 bytes from 52.233.201.66: icmp_seq=3 ttl=112 time=19.6 ms
64 bytes from 52.233.201.66: icmp_seq=4 ttl=112 time=19.2 ms
64 bytes from 52.233.201.66: icmp_seq=5 ttl=112 time=19.9 ms
64 bytes from 52.233.201.66: icmp_seq=6 ttl=112 time=20.3 ms
^C
--- sdu.dk ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5006ms
rtt min/avg/max/mdev = 17.436/19.330/20.282/0.908 ms
jruoho@adorno~$
```

Figure 7: The Classical Unix Tool

Availability Tactics: Detect $\mapsto$ Ping and Echo (2/2)

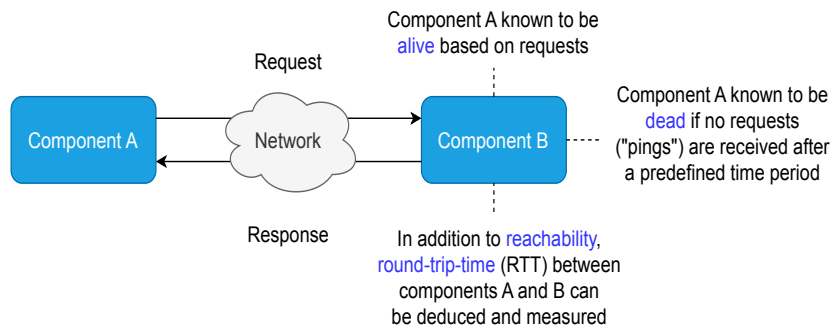
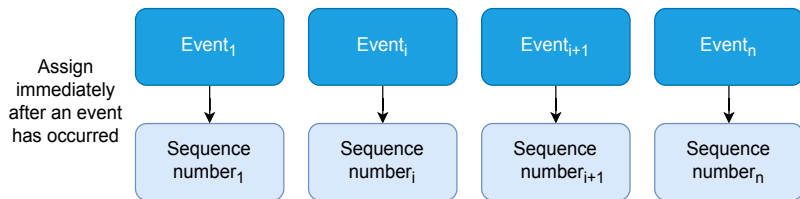


Figure 8: The Classical Unix Tool Extended

Availability Tactics: Detect $\mapsto$ Timestamps



Because the sequence numbers (or timestamps) are monotonically increasing, **inconsistencies** are easy to detect; e.g., $\text{Sequence number}_i > \text{Sequence number}_{i+1}$.

Figure 9: Why Sequence Numbers Might Be Preferred Over Timestamps?

Availability Tactics: Detect $\mapsto$ Sanity Checking (1/2)

- ▶ In general, **sanity checking** recapitulates what was said the fourth lecture about assertions; specific operations, states, inputs, and outputs are verified at runtime for their “sanity”
- ▶ For instance, a machine or a firewall (in reality, a kernel) receiving a classical (ICMP) **echo request** packet verifies its type (8 bits), checksum (16 bits), identifier (16 bits), sequence number (16 bits), and possibly custom payload (variable length) too
 - If there are mismatches or corruptions, the package is dropped and possibly logged

Availability Tactics: Detect $\mapsto$ Sanity Checking (2/2)

- ▶ A further example of sanity checking is **cyclic redundancy check** (CRC), which is a classical error-correction technique used in hardware and networking for detecting alterations
 - The term redundancy signifies that a **check value** is appended without adding new information
- ▶ If a given check value does not pass a verification check, corrective action (such as a retry) can be used
- ▶ While being about integrity, CRC checks are *not like hashes*
 - Nor should they be used for security purposes

Availability Tactics: Detect $\mapsto$ Voting (1/9)

- ▶ **Voting** (for a lack of a better term) involves comparing computational outputs from multiple components that should produce the exact same output, and if they are not producing the same output, deciding which output to use
- ▶ Examples of voting tactics include:
 1. **Replication** is the simplest technique: components are exact copies (replicas) of each other, and, hence, if one component fails, an output from any other component can be used
 - In addition to a performance trade-off (due to overhead), replication cannot protect against design or implementation errors due to a lack of diversity between the components

Availability Tactics: Detect $\mapsto$ Voting (2/9)

2. **Functional redundancy** adds diversification; although functionally diversified components should still produce the exact same output for the same input, they may perform also other function beyond those used for voting
3. **Analytical redundancy** allows diversity in both inputs and outputs; for instance, there are multiple ways to compute an altitude (sensors, radar altimeter, analytical geometry, etc.)
 - Sometimes it is a good idea to cast votes among diverse candidates also in engineering!

Availability Tactics: Detect $\mapsto$ Voting (3/9)

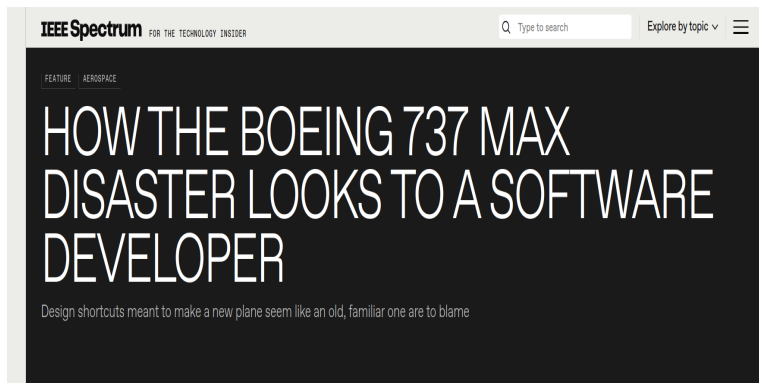


Figure 10: [https://spectrum.ieee.org/
how-the-boeing-737-max-disaster-looks-to-a-software-developer](https://spectrum.ieee.org/how-the-boeing-737-max-disaster-looks-to-a-software-developer)

Availability Tactics: Detect $\mapsto$ Voting (4/9)

- ▶ Obviously, **complexity** increases with functional and particularly analytical redundancy and, therefore, ensuring **consistency between instances** can be hard
- ▶ When considering **performance** in addition to availability, an entirely different question is *how to distribute work* (load) among the components in a system
 - Again, complexity increases substantially also regarding performance when using diversified redundancy techniques

Availability Tactics: Detect $\mapsto$ Voting (5/9)

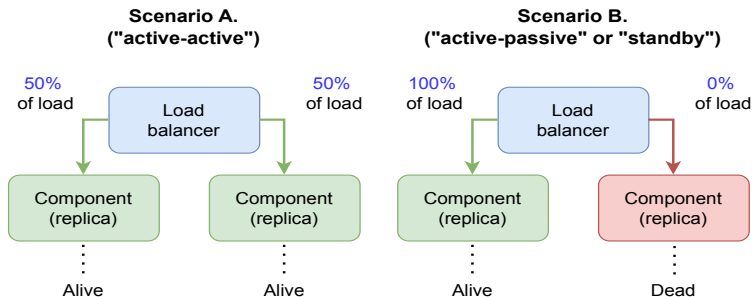


Figure 11: A Simple Example of Load Balancing

Availability Tactics: Detect $\mapsto$ Voting (6/9)

APNIC

LOG IN

Home

Q

≡

From roots to reach: Network resilience in natural disasters

By [Md Main Uddin Hasan](#) on 6 Feb 2026

Category: [Tech matters](#)

Tags: [Bangladesh](#), [community networks](#), [Disaster recovery](#), [DTN](#), [Guest Post](#)

[Tweet](#)

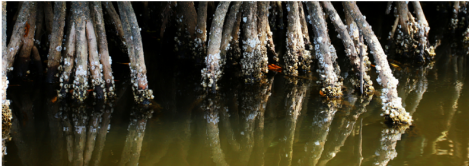


Photo by [Jimmy Blackwell](#) on [Unsplash](#).

Disasters rarely announce themselves through collapsed buildings alone. They are also felt through absence, the sudden silence that follows when communication networks fail. In those early hours, before the scale of physical damage is fully understood, it is often the breakdown of connectivity that determines how broad the impact of a crisis becomes.

Figure 12: <https://blog.apnic.net/2026/02/06/from-roots-to-reach-network-resilience-in-natural-disasters/>

Availability Tactics: Detect $\mapsto$ Voting (7/9)

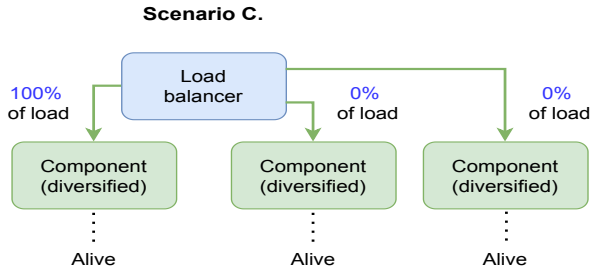


Figure 13: Where and Why Might This Scenario Occur?

Availability Tactics: Detect $\mapsto$ Voting (8/9)

- ▶ A further concept is **data redundancy**; typically, data or parts of it are replicated across components and checksumming is used to detect corruptions
 - The application range from memory, disk drives, and file systems to databases and their tables
- ▶ At the level of hardware, **dual modular redundancy** or **triple modular redundancy** are often used
 - These are essential about low-level replication; for instance, there may be two or three identical logical gates computing the exact same Boolean operator

Availability Tactics: Detect $\mapsto$ Voting (9/9)

Table 2: Majority Gate

Row	Input ₁	Input ₂	Input ₃	Output
1	0	0	0	?
2	0	1	0	?
3	0	0	1	?
4	0	1	1	?
5	1	0	0	?
6	1	0	1	?
7	1	1	0	?
8	1	1	1	?

- Which rows pass the gate (i.e., output 1)?

Availability Tactics: Detect $\mapsto$ Exception Detection

- ▶ By exception detection, we go beyond programming languages
- ▶ At least the following can be theoretically distinguished:
 1. **System exceptions** are dependent on hardware and CPUs; the examples include division by zero and segmentation faults
 2. **Parameter fences** involve placing known patterns (e.g., 0xDEADBEEF) immediately after a variable-length parameter to allow runtime detection of overwrites
 3. **Parameter typing** refers to concrete classes implementing type-length-values specified in an abstract class; it can be used to enforce type safety even in languages without strong typing
 4. **Timeout** refers to a pattern already elaborated; an exception (or interrupt, etc.) is raised in case a component detects that another component has failed to meet its timing requirements

Availability Tactics: Detect $\mapsto$ Self-Tests (1/2)

- ▶ A **self-test** is a tactic that a machine (or a set of machines) either triggers periodically on its own or that is being triggered by another machine acting as a system monitor
- ▶ Something like periodic **virus scans** would be a good example
- ▶ However, self-tests can also related to **maintenance**; for instance, many Linux/Unix either run by default or can be configured to run regular checks about configuration consistency and related integrity checks, which are also used due to software updates discussed in the fourth lecture

Availability Tactics: Detect $\mapsto$ Self-Tests (2/2)

security.conf(5) - NetBSD Manual Pages

COMMAND:
security.conf

SECTION:
5 - File Formats

ARCH:
NONE

COLLECTION:
NetBSD-current

Show

SECURITY.CONF(5) NetBSD File Formats Manual SECURITY.CONF(5)

NAME

security.conf -- daily security check configuration file

DESCRIPTION

The **security.conf** file specifies which of the standard */etc/security* services are performed. The */etc/security* script is run, by default, every night from */etc/daily*, on a NetBSD system, if configured to do so from */etc/daily.conf*.

The variables described below can be set to "NO" to disable the test:

check_entropy	This checks whether the system has enough entropy (see entropy(7)).
check_passwd	This checks the <i>/etc/master.passwd</i> file for inconsistencies.
check_group	This checks the <i>/etc/group</i> file for inconsistencies.

Figure 14: <https://man.netbsd.org/security.conf.5>

Availability Tactics: Repair $\mapsto$ Redundancy (1/3)

- ▶ Redundancy, whether through replicated or diversified components, is the classical repairing tactic
 - To recall: if one fails, others can step in
- ▶ However, achieving redundancy even with replicas can be challenging due to difficulties to ensure **data consistency**
- ▶ Four consistency types can be enumerated:
 1. If high availability is required, **eventual consistency** can be guaranteed: whenever there are no updates, all replicated are merely assumed to eventually converge to the same data state

Availability Tactics: Repair $\mapsto$ Redundancy (2/3)

2. **Quorum consistency** resembles the earlier take on majority voting: an update succeeds only in case a quorum (such a majority or 1/3:th of components) acknowledges the update
 - Something akin the Observer pattern could be used
 - Although *polling* is often discouraged – and often for a good reason, it too might be suitable in some contexts
3. **Synchronized consistency** allows simultaneous writes at multiple data components but requires conflict resolution to handle concurrent updates
4. **Read-after-write consistency** means that all subsequent reads after a write reflects that particular write, regardless whichever replica handles the read operation

Availability Tactics: Repair $\mapsto$ Redundancy (3/3)

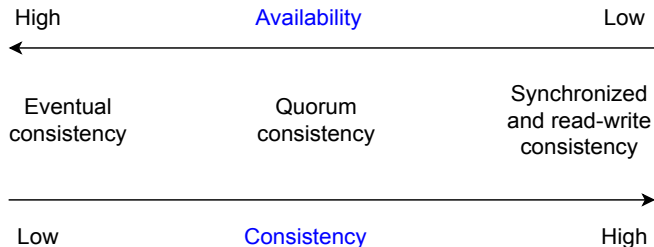


Figure 15: A Summary of the Consistency Types

Availability Tactics: Repair $\mapsto$ Rollbacks and Updates

- ▶ Rollbacks were introduced already in the previous lecture, but to recall: upon a failure is detected, a system can be rolled back to a known good state
 - But **what is a known good state**? If cyber security is considered, the answer is unfortunately often speculative
- ▶ • As always, there are also trade-offs; among them questions such as **how far back** should states be stored and **how much is lost** if rolling back to a faraway state?
 - These are classical questions stemming from **backups**
- ▶ Upgrades too were discussed during the fourth lecture

Availability Tactics: Repair $\mapsto$ Exception Handling (1/2)

- ▶ When caught, exceptions should be obviously handled
- ▶ Although it may sound easy, it is often not easy at all
- ▶ When considering either large distributed systems or specialized machines, careful thinking and designing about some **fail-safe** solutions are typically needed
 - For instance, **what should a drone do** if it loses a connection to its controller?

Availability Tactics: Repair $\mapsto$ Exception Handling (2/2)

KASSERT(9)

Kernel Developer's Manual

KASSERT(9)

NAME

KASSERT -- kernel expression verification macros

SYNOPSIS

options **INVARIANTS**

```
#include <sys/param.h>
```

```
#include <sys/system.h>
```

```
KASSERT(expression, msg);
```

```
MPASS(expression);
```

DESCRIPTION

Assertions are widely used within the FreeBSD kernel to verify programmatic assumptions. For violations of run-time assumptions and invariants, it is desirable to fail as soon and as loudly as possible. Assertions are optional code; for non-recoverable error conditions an explicit call to *panic*(9) is usually preferred.

Figure 16: <https://man.freebsd.org/cgi/man.cgi?query=KASSERT>

Availability Tactics: Repair $\mapsto$ Retry (1/3)

- ▶ When there is a reason to believe that a fault might be only transient, the **retry** tactic is a sensible choice

```
while i < max_retries; do

    rv = compute()

    if rv == SUCCESS
        return OK

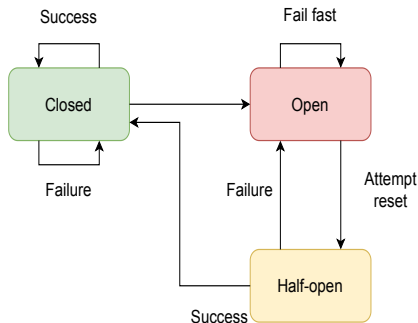
    sleep(delay)
    i++

return FAULT
```

Availability Tactics: Repair $\mapsto$ Retry (2/3)

- ▶ The previous simple retry example could be improved with **exponential back-off**, meaning that the sleeping delay between calls would increase as failures increase
- ▶ Another improvement might be a **circuit breaker** pattern: instead of returning FAULT after `max_retries` is reached, the routine would wait longer and then try the loop again, perhaps only returning FAULT after a few such iterations
 - To avoid wasting resources, the rationale is to **fail fast**
- ▶ Sometimes it also a good idea to add **timeouts** to all availability-specific calls (memory, networking, etc.)
- ▶ **Caching** is a good idea too; it is covered in the next lecture

Availability Tactics: Repair $\mapsto$ Retry (3/3)



Closed: requests are passed to a service and faults increase counters. If a counter exceed a threshold, the circuit breaker is tripped and transitions into the open state.

Open: requests are not passed to a service. Instead, all faults are passed to clients immediately. If responsiveness is again gained or ping-echos indicate so, the circuit breaker transitions into the half-open state.

Half-open: only a limited number of requests are passed to a service. If a service responds with successes, the circuit breaker transitions again into the closed state. But if requests still fail, the circuit breaker transitions back to the open state.

Figure 17: A State Diagram for the Circuit Breaker Pattern (adopted from Montesi & Weber 2016, Fig. 1 and p. 2)

Availability Tactics: Repair $\mapsto$ Ignore

- ▶ While it is possible (but perhaps not sensible) to sometimes use the **ignore** tactic for basically just accepting a probability of a failure, there are many cases where ignoring via sanity checking is reasonable; cf.

```
x = read_sensor(sensor_103)
```

```
if x > 0
```

```
    return OK
```

```
return FAULT
```

Availability Tactics: Repair $\mapsto$ Degrade and Reconfigure

- ▶ The **graceful degradation** tactic seeks to maintain the *availability of most critical* functions while degrading, suspending, or outright dropping less critical functions
 - For instance, in the face of a denial of service attack against a public transportation system, a payment system might be maintained at the expense of maps and related auxiliary things
- ▶ However, it is possible to also use the graceful degradation without ties to critical functions; e.g., serve the *majority* by denying requests for the minority; only serve some particular *geographic* locations; *compute work in progress first* without accepting new requests; etc.

Availability Tactics: Repair $\mapsto$ Reconfigure

- ▶ The **reconfiguration** tactic is about recovering from failures through reassigning responsibilities, loading alternative configurations, or something analogous
 - For instance, a server under denial of service might reload a configuration that drops HTTP but still allows SSH
- ▶ A good example would be a reconfiguration that **slows things down** once some critical threshold is reached
 - For instance, CPUs throttle them in case of overheating

Availability Tactics: Reintroduce $\mapsto$ Shadow

- **Reintroduction** occurs when a failed component is reintroduced after it has been repaired

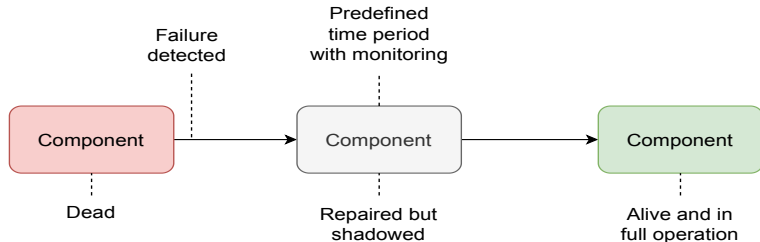


Figure 18: The Shadow Tactic in a Nutshell

Availability Tactics: Reintroduce $\mapsto$ State Re-sync.

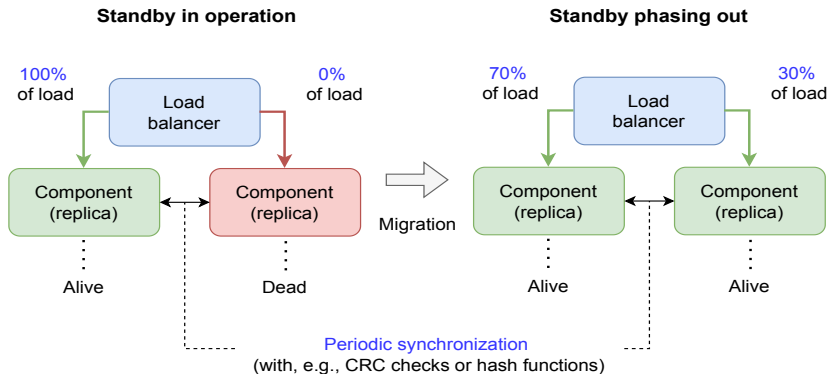


Figure 19: An Illustration of State Re-synchronization

Availability Tactics: Reintroduce $\mapsto$ Escalating Restart

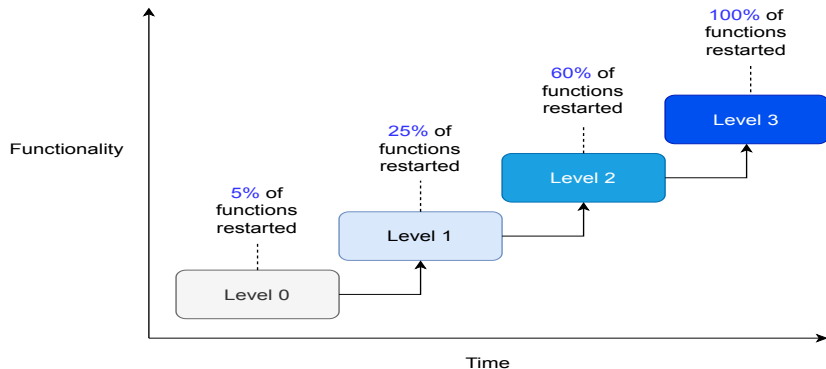


Figure 20: An Illustration of Escalating Restart

Availability Tactics: Reintroduce $\mapsto$ Nonstop Forwarding

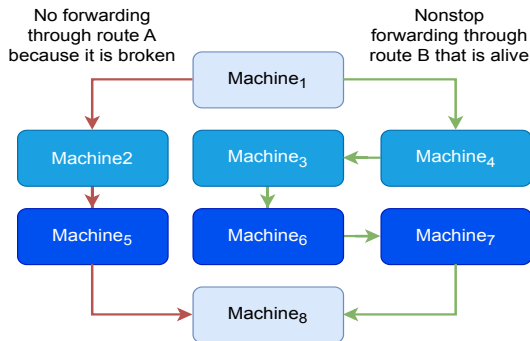


Figure 21: An Illustration of Nonstop Forwarding

Availability Tactics: Prevent $\mapsto$ Isolate (1/4)

- ▶ **Isolation** is a general tactic for both security and availability
- ▶ Recall the notion of **blast radius** from the fourth lecture
 - If machines and their components, or some of them, are isolated from each other, a blast radius may not cover the entire system; important machines or components may still work even in case most are down
 - A concept of **sharding** is often used in this context
- ▶ The blast radius concept correlates with a more general concept of **cascading failures**; a single failure somewhere in a large system may – in the worst case scenario – cascade into the whole system, possibly taking it down

Availability Tactics: Prevent $\mapsto$ Isolate (2/4)

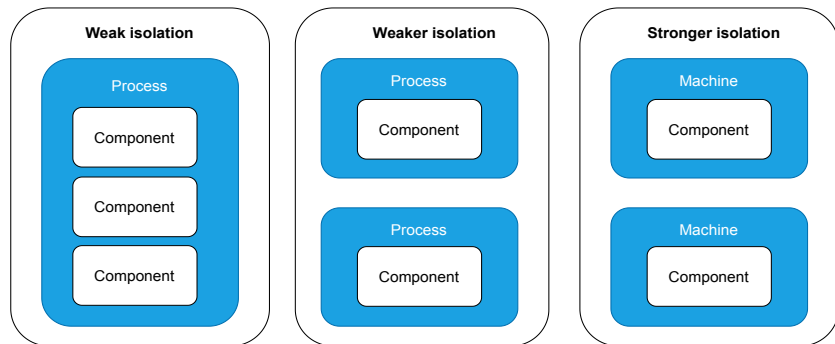


Figure 22: Recall the Points from the First Lecture
("Isolation Is Related to Coupling but Not Equal to It")

Availability Tactics: Prevent $\mapsto$ Isolate (3/4)

- ▶ When designing isolation, Adkins et al. (2020) recommend **functional separation of different failure domains**
 - The goal is to minimize a probability that a failure does not affect all components of a system at once
- ▶ They further emphasize **data isolation**; each failure domain needs its own data to be functionally independent
 - The goal is to minimize the propagation of corrupted or otherwise bad data from one failure domain to another
 - Note that “data” should be understood broadly here; it is a good idea to write also known good configurations to disk

Availability Tactics: Prevent $\mapsto$ Isolate (4/4)

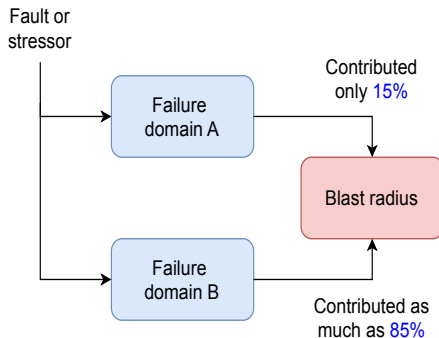


Figure 23: The Ideas of A/B Testing Apply Also to Reliability Engineering

Availability Tactics: Prevent $\mapsto$ Removal and Transactions

- ▶ The **removal** tactic essential recapitulates the general wisdom of redundancy; seek to ensure that a failed component can be safely removed altogether (and replaced if necessary)
- ▶ The tactics related to **transactions** build upon the classical but difficult ACID principle; seek to ensure atomicity, consistency, isolation, and durability

```
ATOMIC(9)                                Kernel Developer's Manual                                ATOMIC(9)

NAME
    atomic_add,      atomic_clear,      atomic_cmpset,      atomic_fcmpset,
    atomic_fetchadd,      atomic_interrupt_fence,      atomic_load,
    atomic_readandclear,      atomic_set,      atomic_subtract,      atomic_store,
    atomic_thread_fence -- atomic operations

SYNOPSIS
    #include <machine/atomic.h>
```

Figure 24: <https://man.freebsd.org/cgi/man.cgi?query=atomic>

Availability Tactics: Prevent $\mapsto$ Predictive Models

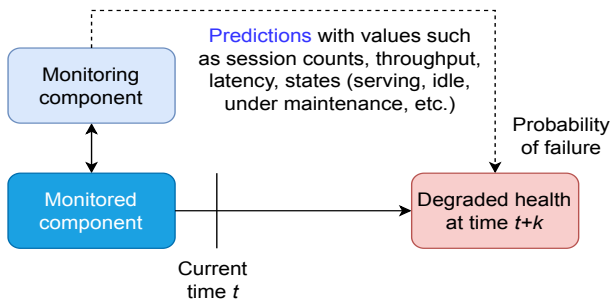


Figure 25: An Naïve Illustration of Predictive Maintenance

Availability Tactics: Prevent $\mapsto$ Exception Prevention

- ▶ Of course, regarding exceptions, the best solution is to try to prevent them from firing to begin with
- ▶ However, it is difficult to provide universal guidelines on how to do that; the examples range from **error-correction codes** (CRC checks are a simple example here) used in networking and hardware to **smart pointers** in C++
- ▶ In general, a sensible guideline regarding programming is to avoid complex try-catch exception chains and generally **minimize exception propagation** chains

Availability Tactics: Prevent $\mapsto$ Increase Competence (1/2)

- ▶ We do generally endorse the KISS principle but sometimes it is not a good idea; oftentimes, *slightly* increasing complexity with fault-tolerance functionality is a good idea
- ▶ For instance, a poorly designed module needing a network might throw an exception (or worse) in case a given network happens to be down
 - By slightly increasing competence of the module, it could be redesigned to put itself into an idle state and wait for the network to come back
- ▶ But recall also the earlier point that sometimes it is better to also crash and burn loudly; hence, there are again **trade-offs**

Availability Tactics: Prevent $\mapsto$ Increase Competence (2/2)

DESCRIPTION

`kmem_alloc()` allocates kernel wired memory. It takes the following arguments.

size Specify the size of allocation in bytes.

kmflags Either of the following:

KM_SLEEP If the allocation cannot be satisfied immediately, sleep until enough memory is available. If **KM_SLEEP** is specified, then the allocation cannot fail.

KM_NOSLEEP Don't sleep. Immediately return NULL if there is not enough memory available. It should only be used when failure to allocate will not have harmful, user-visible effects.

Use of `KM_NOSLEEP` is strongly discouraged as it can create transient, hard to debug failures that occur when the system is under memory pressure.

In situations where it is not possible to sleep, for example because locks are held by the caller, the code path should be restructured to allow the allocation to be made in another place.

Figure 26: <https://man.netbsd.org/kmem.9>

Availability Challenges

- ▶ While most what was said is easy on paper but difficult in practice, **maintaining state** is perhaps the most frequently encountered difficulty with availability and fault-tolerance
 - Therefore, if possible, seek to prefer **stateless designs**, including microservices and RESTful applications
- ▶ When moving beyond the Web and current hyperscalers, however, stateful systems are still often necessary
 - Therefore, a few further points about availability patterns

Availability Patterns: Active Redundancy (1/2)

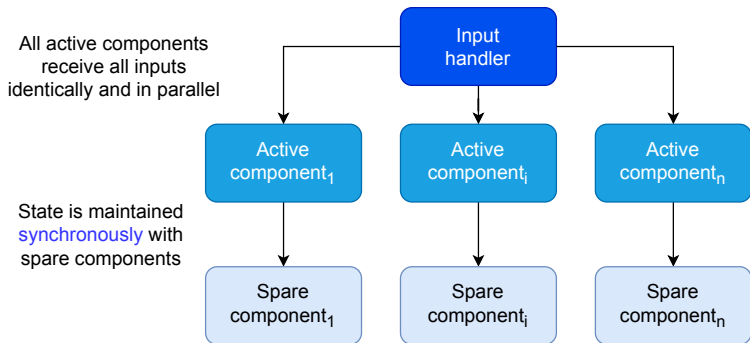


Figure 27: An Illustration of Active (“Hot Spare”) Redundancy

Availability Patterns: Active Redundancy (2/2)

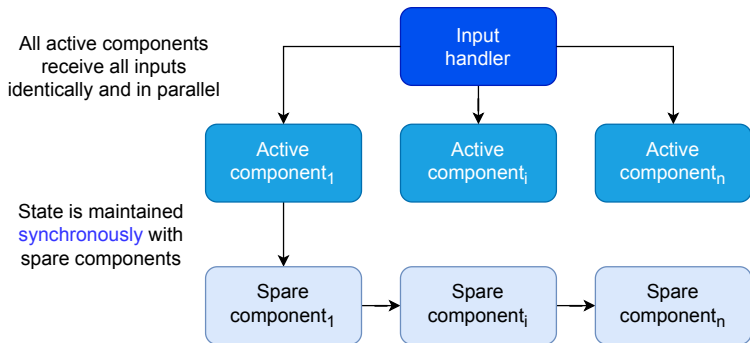


Figure 28: Since State Is Maintained, There Are Many Design Possibilities

Availability Patterns: Data Redundancy (1/2)

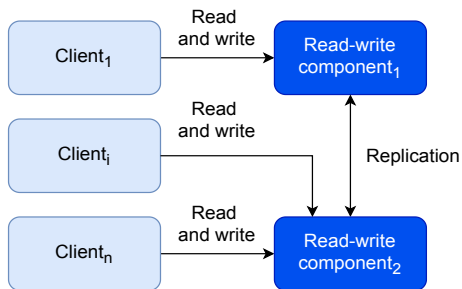


Figure 29: Active Data Redundancy #1

Availability Patterns: Data Redundancy (2/2)

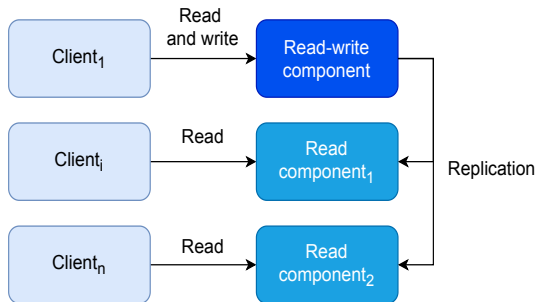


Figure 30: Active Data Redundancy #2: **How It Helps Availability?**

Availability Patterns: Passive Redundancy

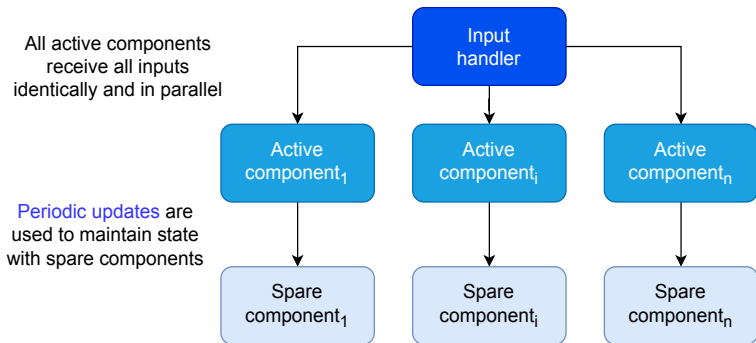
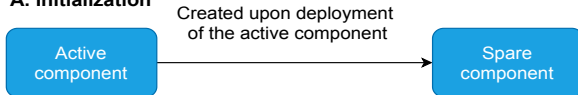


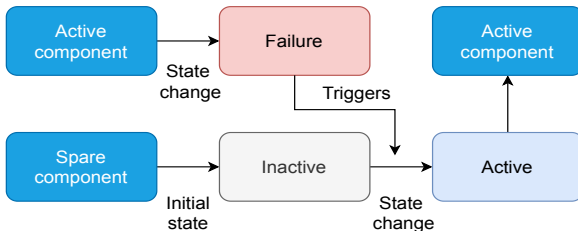
Figure 31: An Illustration of Passive ("Warm Spare") Redundancy

Availability Patterns: Spare Redundancy

A. Initialization



B. Operation



(A monitoring component required but not shown for brevity)

Figure 32: An Illustration of Spare (or “Cold Spare”) Redundancy

Availability Patterns: FIFO and LIFO Queueing

- ▶ Sometimes, a **LIFO** (“last in, first out”) queueing is better than **FIFO** (“first in, first out”) queueing
 - A FIFO processes requests as they arrive, whereas LIFO prioritizes more recent requests
 - For instance, using a FIFO can be useful to avoid scenarios where a large amount of clients are about to retry their requests; the retries themselves eat up resources even more

Availability Patterns: Equitable Resource Allocation

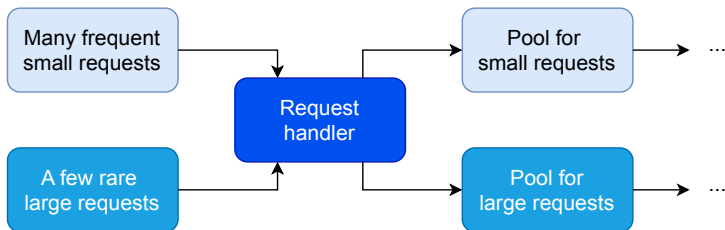


Figure 33: An Illustration of Equitable Resource Allocation (sometimes also known as the “bulkheads pattern”)

Availability Patterns: Dependencies

- ▶ Adkins et al. (2020) recommend consider dependencies when designing for availability (here, recall the points from the third lecture about different dependency concepts)
- ▶ For instance, redundancy could be engineered via **isolating critical but low-dependency components**; these are those particular components that have only *minimal dependencies to failure domains* of a system
- ▶ In addition, they give two practical advices:
 1. “Use data cached on local storage to avoid depending on a remote database” (cf. versus Supabase, anyone...?)
 2. With some components, deploy first with “older code and configs to avoid recent bugs in newer versions.”

Availability Patterns: Taints and Quarantines

- ▶ A further pattern, which is related also to security, is about **tainting**: whenever a corruption is detected, the corrupted data should not be used and no actions should be derived from the corrupted data
- ▶ For these reasons too, the **data isolation** is important
- ▶ Yet, it is a good idea to **quarantine** the corrupted data
 - This is what for instance file systems and hard drives do

Availability Patterns: Saga (1/3)

- ▶ The **saga pattern** divides transaction processing into local transactions: each local transaction completes its action atomically within a single component (or service)
 - It is similar to the *batch-sequential pattern* from the third lecture but with an addition of error correction
- ▶ Each local transaction updates its own database and notifies the subsequent local transaction about its completion
- ▶ The pattern is good for maintaining consistency of data, but complexity increases because if a local transaction fails, corrective transactions must be propagated

Availability Patterns: Saga (2/3)

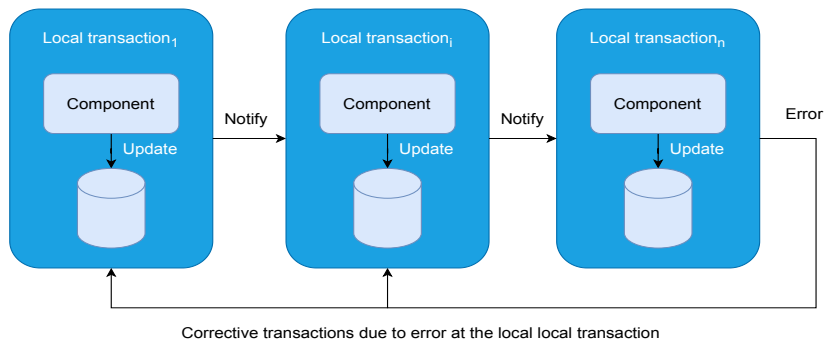


Figure 34: An Illustration of the Saga Pattern

Availability Patterns: Saga (3/3)

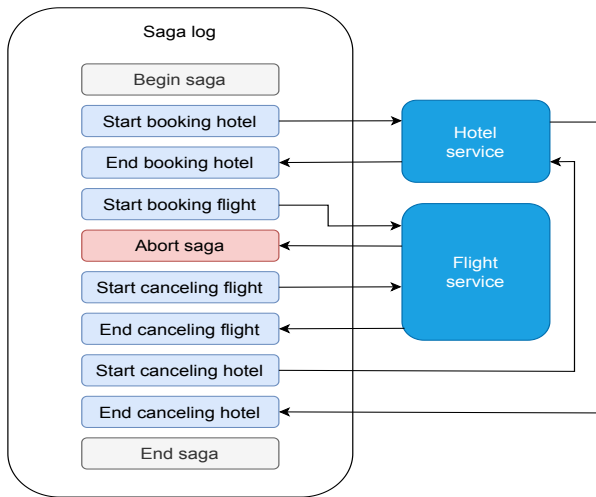


Figure 35: An Illustration of Saga Failures (from Dürr et al. 2021, Fig. 1)

Availability Patterns: A Summary

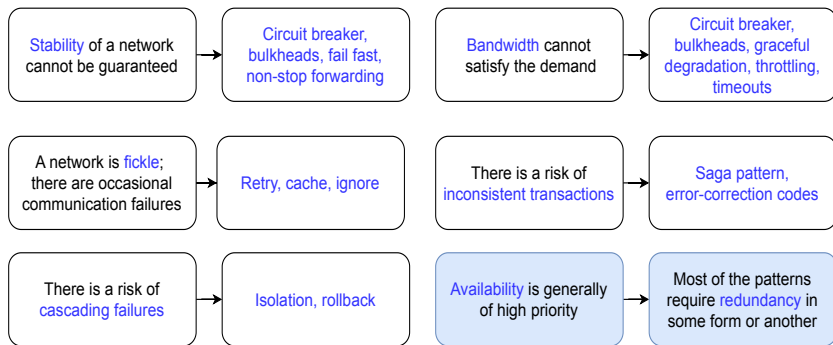


Figure 36: Even Though It Is Difficult to Provide Universal Guidelines, Some of the Patterns Correlate With Different Scenarios

The Maintainability and Cost Trade-Offs

“Today, systems often degrade without clear signals. Traces look fine, dashboards stay green, and yet performance quietly erodes. This shift is not just about scale. It reflects how modern systems accumulate complexity long before they actually need it.”

“Teams respond by designing for edge cases they may never encounter. They build resilience against hypothetical traffic patterns while ignoring the operational cost of running the system today.”

“Services exist primarily to justify their own abstractions. Configuration replaces code, and behavior is distributed across files, dashboards, and runtime flags. Understanding the system requires navigating documentation that is already out of date The architecture technically works, but only because people avoid touching it.”

(quoted from Williams 2026)

Questions?

Case #5

- ▶ Although no longer maintained, Hystrix was at one point a fault-tolerance framework used by Netflix
- ▶ Study the framework and its documentation (<https://github.com/Netflix/Hystrix>)
 - What were their **scenarios**?
 - What **tactics** they used?
 - How about **patterns**?

Thank you

Questions?

Additional References

1. Ruohonen, J & Hjerpe, K. & von Zastrow, M. (2024): An Exploratory Case Study on Data Breach Journalism, ARES 2024, pp. 1–9, ACM, <https://dl.acm.org/doi/pdf/10.1145/3664476.3670456>
2. Ruohonen, J & Hyrynsalmi, S. & Leppänen, V. (2015): The Sigmoidal Growth of Operating System Security Vulnerabilities: An Empirical Revisit. *Computers & Security* 55, pp. 1–20., <https://doi.org/10.1016/j.cose.2015.07.001>
3. Montesi, F. & Weber, J. (2016): Circuit Breakers, Discovery, and API Gateways in Microservices. <https://arxiv.org/pdf/1609.05830>
4. Dürr, K. & Lichtenthäler, R. & Wirtz, G. (2021): An Evaluation of Saga Pattern Implementation Technologies. ZEUS Workshop, pp. 1–9, <https://ceur-ws.org/Vol-2839/paper12.pdf>
5. Williams, A. (2026): How Over-Engineering Became the New Technical Debt in Distributed Systems. Communications of the ACM, <https://cacm.acm.org/blogcacm/how-over-engineering-became-the-new-technical-debt-in-distributed-systems>